

DANESHVAR AMROLLAHI

✉ daneshvar@cs.stanford.edu  cs.stanford.edu/~daneshva  github.com/daneshvar-amrollahi
389 Jane Stanford Way, CoDa #W310, Stanford, CA 94305, United States

EDUCATION

- **Stanford University** 2024/01 - Present
PhD in Computer Science Advisor: Clark Barrett
- **University of Tehran** 2018/09 - 2023/02
BS in Computer Engineering (Software) GPA: 18.02/20.00

SELECTED PUBLICATIONS

- D. Amrollahi et al. **AI Coding Benchmarks Need Proofs, Not Just Tests.** *Under review at Neural Information Processing Systems (NeurIPS 2026) Position Paper Track.*
- B. Miranda et al. **VeriBench: End-to-End Formal Verification Benchmark for AI Coding Agents in Lean 4.** *Under review at Neural Information Processing Systems (NeurIPS 2026) Datasets and Benchmarks Track.*
- O. Wang et al. **Anonymized.** *Under review at Conference on Language Modeling (COLM 2026).*
- D. Amrollahi et al. **Faithful Autoformalization via Roundtrip Verification and Repair.** *Under review at ACL Rolling Review (ARR).*
- C. Sun et al. **VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus.** *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*
- D. Amrollahi et al. **Towards SMT Solver Stability via Input Normalization.** *Formal Methods in Computer-Aided Design (FMCAD 2025).*
- D. Amrollahi et al. **Solving Invariant Generation for Unsolvable Loops.** *International Static Analysis Symposium (SAS 2022).* **Awarded the Radhia Cousot Young Researcher Best Paper Award.**

INDUSTRY EXPERIENCE

- **PhD Software Engineer Intern at Uber Technologies, Inc.** Seattle, United States
Programming Systems Team 2026/06 – 2026/09
Building LLM-driven developer tools that use **LLMs** augmented with **program analysis** for automated bug detection, code repair, and program synthesis across Uber's large-scale production codebase.
- **Applied Science Intern at Amazon Web Services (AWS)** Santa Clara, United States
Automated Reasoning Group 2025/06 – 2025/09
Built a tool from scratch for validating SMT-LIB arithmetic queries and models: integrated a Rust parser to construct Lean objects, developed a Rust–Lean FFI pipeline, encoded SMT terms and semantics in **Lean**, and formally proved correctness of optimized evaluation against reference semantics.

RESEARCH EXPERIENCE

- **PhD Student at Center for Automated Reasoning, Stanford University** Stanford, United States
Under Prof. Clark Barrett 2024/01 – Present
 1. Proposed a **roundtrip verification** and **repair** framework for **faithful autoformalization** of natural-language into first-order logic, without ground-truth annotations.
 2. Co-developed **Lean** formalizations of Python programs and curated tasks for **VeriBench**, an end-to-end formal verification benchmark for evaluating **LLMs** on verified code generation.
 3. First-authored a position paper arguing that **AI coding benchmarks** need **machine-checkable proofs**, not just tests, to reliably evaluate code generated by **AI coding agents**.
 4. Solo-led the design, implementation, and large-scale evaluation of a **normalization** algorithm in the **cvc5** industrial SMT solver. Proved the problem as hard as **graph isomorphism** and gave a practical approximation, evaluated across $\sim 570K$ solver runs, significantly improving stability of **cvc5** and **Z3**.

- **Research Intern at Dependable Systems Lab, EPFL**

Lausanne, Switzerland

Under Prof. George Candea

2022/07 – 2022/08

Extended the KLEE **symbolic execution** engine to generate and track **quantified formulas** in **Z3**, enabling **loop summarization** to mitigate **path explosion** and improve scalability in analyzing **network software**. Involved substantial C++ **systems programming** and **SMT solver** integration.

- **Research Intern at Automated Program Reasoning Group, TU Wien**

Vienna, Austria

Under Prof. Laura Kovács and Prof. Ezio Bartocci

2021/07 - 2022/02 + 2023/05 - 2023/12

Developed a **program synthesis** system for recursive programs grounded in **theorem proving**, and a **static analysis** tool for synthesizing loop invariants using **symbolic reasoning** approaches.

HONORS AND AWARDS

- **Radhia Cousot Young Researcher Best Paper Award**

2022

29th Static Analysis Symposium (SAS 2022)

- **Stanford School of Engineering (SoE) Fellowship**

2024

Awarded a \$12900/quarter stipend and full tuition coverage for one year

- **Ranked 8th in ACM-ICPC (International Collegiate Programming Contest)**

2020

West Asia Region, Tehran site

- **Summer@EPFL Fellowship**

2022

Ranked top 1.5% among 4000 applicants and awarded a 1600 CHF/month fellowship over summer

PROJECTS

- **cvc5** — github.com/cvc5/cvc5

C++

Implemented an efficient and scalable normalization pre-processing pass in this industrial-strength SMT solver for achieving performance stability across semantics-preserving query mutations.

- **KLEE** — github.com/bolt-perf-contracts/klee/pull/9

C++, Z3

Integrated Z3's quantified reasoning into KLEE's symbolic execution engine to summarize loops and mitigate path explosion, improving scalability on complex programs involving heavy string operations.

- **ARM Processor** — github.com/daneshvar-amrollahi/ARM

Verilog, ModelSim, Quartus

Implemented a comprehensive 5-stage pipelined ARM processor with data forwarding, hazard detection, cache memory, and SRAM controller.

- **Vampire** — github.com/vprover/vampire

C++

Implemented a framework within this theorem prover for synthesizing verified recursive programs.

- **Polar** — github.com/probing-lab/polar

Python, SymPy

Implemented a polynomial loop-invariant synthesizer for (probabilistic) unsolvable loops.

- **Koloocheh** — github.com/daneshvar-amrollahi/Koloocheh

Python, gRPC

A peer-to-peer file-sharing system employs the flooding algorithm for search operations and ensures a small graph diameter by leveraging random graph properties.

- **xv6** — github.com/daneshvar-amrollahi/os-lab-xv6

C

Extended xv6 operating system kernel with several extra features such as various new system calls, multilevel queue scheduling, and synchronization.

SKILLS

- **Programming Languages:**

- Experienced in C, C++, Python.
- Familiar with Rust, Go, Scala, Bash, Verilog, SystemVerilog.

- **Tools:** cvc5, Z3, Lean, KLEE, L^AT_EX.

- **Academic (Sub)review:** ISSAC 2024, TACAS 2026, Journal of Symbolic Computation.